SLAM Over Network with CycloneDDS

Complete Configuration Guide

Table of Contents

1. CycloneDDS Configuration
2. SLAM Toolbox Network Optimization
3. Launch File Modifications
4. QoS Settings for SLAM Topics
5. Network Architecture Setup
6. Bandwidth Optimization
7. Network Testing
8. Firewall Configuration
9. Troubleshooting

---

1. CycloneDDS Configuration

Create `~/cyclonedds.xml`:

xml

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<CycloneDDS xmlns="https://cdds.io/config"
            xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
            xsi:schemaLocation="https://cdds.io/config
            https://raw.githubusercontent.com/eclipse-cyclonedds/cyclonedds/master/etc/cyclonedds.xsd">
  <Domain id="any">
    <General>
      <!-- Increase fragment size for large messages (maps, point clouds) -->
      <FragmentSize>64KiB</FragmentSize>
      <MaxMessageSize>65500B</MaxMessageSize>

      <!-- Network optimization -->
      <AllowMulticast>true</AllowMulticast>
      <EnableMulticastLoopback>true</EnableMulticastLoopback>
    </General>

    <Internal>
      <!-- Increase buffer sizes for SLAM data -->
      <SocketReceiveBufferSize min="10MB"/>
      <SocketSendBufferSize min="10MB"/>

      <!-- Retransmission settings for reliability -->
      <RetransmitMerging>adaptive</RetransmitMerging>
      <RetransmitMergingPeriod>5ms</RetransmitMergingPeriod>
    </Internal>

    <Discovery>
      <!-- Faster peer discovery over network -->
      <ParticipantIndex>auto</ParticipantIndex>
      <SPDPInterval>1s</SPDPInterval>
    </Discovery>

    <Tracing>
      <!-- Enable for debugging network issues -->
      <Verbosity>warning</Verbosity>
      <OutputFile>stdout</OutputFile>
    </Tracing>
  </Domain>
</CycloneDDS>
```

Key Parameters Explained

Parameter	Purpose	Recommended Value
`FragmentSize`	Size of data chunks sent over network	64KiB
`MaxMessageSize`	Maximum single message size	65500B
`SocketReceiveBufferSize`	Receive buffer for incoming data	10MB
`SocketSendBufferSize`	Send buffer for outgoing data	10MB
`SPDPInterval`	Discovery ping interval	1s

2. SLAM Toolbox Network Optimization

Add these parameters to your `slam_toolbox` configuration:

yaml

```yaml
slam_toolbox:
  ros__parameters:
    # ========== CORE CONFIGURATION ==========
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ========== FRAME CONFIGURATION ==========
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan

    # ========== SLAM MODE ==========
    use_map_saver: true
    slam_mode: true

    # ========== STATUS AND DEBUGGING ==========
    publish_status: true
    status_topic: /slam_status
    debug_logging: false  # Disable for network (reduces data)
    visualize_trajectory: true

    # ========== NETWORK OPTIMIZATION ==========
    # Reduce data volume over network
    throttle_scans: 3  # Process every 3rd scan (reduced from 2)
    map_update_interval: 5.0  # Update map every 5s (increased from 2.0)

    # Increase timeouts for network latency
    transform_timeout: 1.0  # Increased from 0.5 for network delay
    tf_buffer_duration: 60.0  # Increased from 30.0, keep more history

    # Reduce publishing rate to save bandwidth
    transform_publish_period: 0.2  # 5Hz instead of 10Hz

    # Map publishing
    map_publish_period: 10.0  # Publish full map every 10s
    scan_queue_size: 1  # Don't queue scans (drop old ones)

    minimum_time_interval: 0.5

    # ========== MAP PARAMETERS ==========
    resolution: 0.10  # 10cm instead of 5cm (reduces map size by 4x)
    max_laser_range: 80.0
```

```
# ========== MOVEMENT THRESHOLDS ==========
minimum_travel_distance: 0.3
minimum_travel_heading: 0.3

# ========== MEMORY CONFIGURATION ==========
stack_size_to_use: 40000000
enable_interactive_mode: true

# ========== SCAN MATCHING ==========
use_scan_matching: true
use_scan_barycenter: true
scan_buffer_size: 10
scan_buffer_maximum_scan_distance: 10.0
link_match_minimum_response_fine: 0.1
link_scan_maximum_distance: 1.5

# ========== LOOP CLOSURE ==========
do_loop_closing: true
loop_search_maximum_distance: 5.0
loop_match_minimum_response_fine: 0.3
loop_match_minimum_chain_size: 10
loop_match_maximum_variance_coarse: 3.0
loop_match_minimum_response_coarse: 0.35

# ========== CORRELATION PARAMETERS ==========
correlation_search_space_dimension: 0.5
correlation_search_space_resolution: 0.01
correlation_search_space_smear_deviation: 0.1

loop_search_space_dimension: 8.0
loop_search_space_resolution: 0.05
loop_search_space_smear_deviation: 0.03

# ========== SCAN MATCHER TUNING ==========
distance_variance_penalty: 0.5
angle_variance_penalty: 1.0
fine_search_angle_offset: 0.00349
coarse_search_angle_offset: 0.349
coarse_angle_resolution: 0.0349
minimum_angle_penalty: 0.9
minimum_distance_penalty: 0.5
use_response_expansion: true
```

Network-Specific Parameter Changes

Parameter	Default	Network Value	Reason
throttle_scans	2	3	Reduce bandwidth
map_update_interval	2.0	5.0	Less frequent updates
transform_timeout	0.5	1.0	Account for latency
transform_publish_period	0.1	0.2	Lower publish rate
resolution	0.05	0.10	Smaller map size

3. Launch File Modifications

Method 1: Set in Launch File

python

```python
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    # Set CycloneDDS configuration
    os.environ['CYCLONEDDS_URI'] = 'file:///home/your_user/cyclonedds.xml'
    os.environ['RMW_IMPLEMENTATION'] = 'rmw_cyclonedds_cpp'

    # Load SLAM configuration
    slam_config = os.path.join(
        get_package_share_directory('your_package'),
        'config',
        'slam_toolbox.yaml'
    )

    slam_node = Node(
        package='slam_toolbox',
        executable='async_slam_toolbox_node',
        name='slam_toolbox',
        parameters=[slam_config],
        output='screen'
    )

    return LaunchDescription([
        slam_node
    ])
```

Method 2: Set in Terminal

```bash
# Export before launching
export CYCLONEDDS_URI=file://$HOME/cyclonedds.xml
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

# Then launch
ros2 launch your_package slam.launch.py
```

Method 3: Add to ~/.bashrc (Permanent)

```bash
# Edit ~/.bashrc
nano ~/.bashrc

# Add these lines at the end
export CYCLONEDDS_URI=file://$HOME/cyclonedds.xml
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export ROS_DOMAIN_ID=42

# Save and reload
source ~/.bashrc
```

---

4. QoS Settings for SLAM Topics

Add QoS overrides to your SLAM launch file:

```python
from launch_ros.actions import Node

slam_node = Node(
    package='slam_toolbox',
    executable='async_slam_toolbox_node',
    name='slam_toolbox',
    parameters=[slam_config],
    remappings=[
        ('/scan', '/scan'),
    ],
    arguments=[
        '--ros-args',
        '--log-level', 'warn',
        # Override QoS for network reliability
        '-p', 'qos_overrides./scan.reliability:=best_effort',
        '-p', 'qos_overrides./scan.durability:=volatile',
    ],
    output='screen'
)
```

QoS Profiles Explained

QoS Setting	Value	Reason
`reliability`	`best_effort`	Prioritize speed over guaranteed delivery
`durability`	`volatile`	Don't store old messages
`history`	`keep_last`	Only keep most recent data

---

5. Network Architecture Setup

On Robot (Onboard Computer)

```bash
# Set environment variables
export ROS_DOMAIN_ID=42
export CYCLONEDDS_URI=file://$HOME/cyclonedds.xml
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

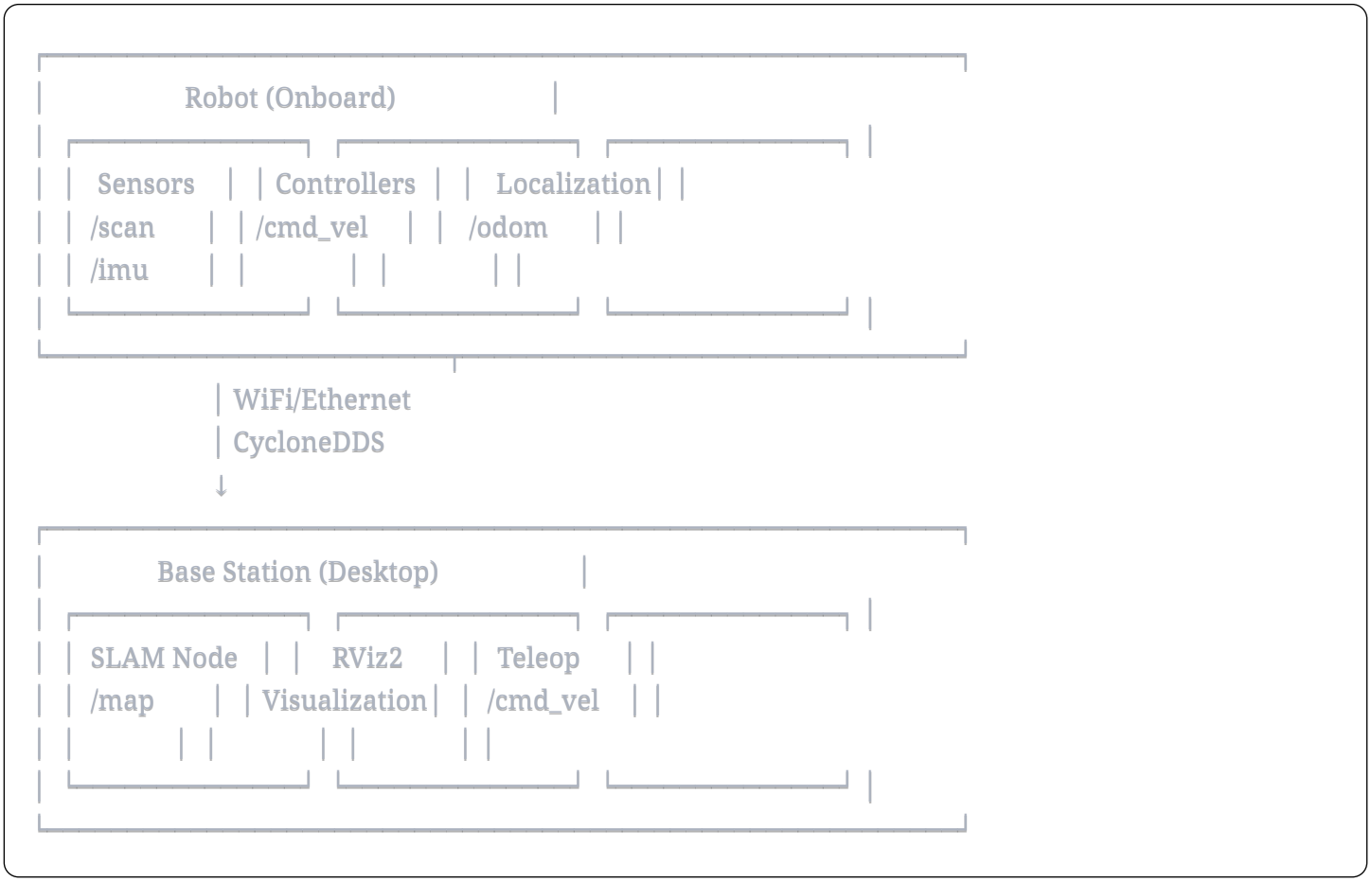
# Launch robot nodes (sensors, controllers)
ros2 launch rover_bringup robot.launch.py
```

On Base Station (SLAM Computer)

```bash
# Set SAME environment variables
export ROS_DOMAIN_ID=42  # MUST MATCH robot
export CYCLONEDDS_URI=file://$HOME/cyclonedds.xml
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

# Launch SLAM and visualization
ros2 launch rover_bringup slam.launch.py
```

Network Diagram



---

6. Bandwidth Optimization

Laser Scan Downsampling

If `/scan` uses too much bandwidth, downsample it:

python

```python
# Add a laser scan filter node
from launch_ros.actions import Node

scan_filter = Node(
  package='laser_filters',
  executable='scan_to_scan_filter_chain',
  parameters=[{
    'scan_filter_chain': [{
      'name': 'box_filter',
      'type': 'laser_filters/LaserScanBoxFilter',
      'params': {
        'box_frame': 'base_link',
        'min_x': -10.0,
        'max_x': 10.0,
        'min_y': -10.0,
        'max_y': 10.0,
      }
    }, {
      'name': 'speckle_filter',
      'type': 'laser_filters/LaserScanSpeckleFilter',
      'params': {
        'filter_type': 1,
        'max_range': 80.0,
        'max_range_difference': 1.0,
      }
    }]
  }],
  remappings=[
    ('scan', 'scan_raw'),
    ('scan_filtered', 'scan')
  ]
)
```

Image Compression

For camera data over network:

python

```python
# Use compressed image transport
image_transport = Node(
    package='image_transport',
    executable='republish',
    name='image_republisher',
    arguments=[
        'raw',
        'compressed',
        '--ros-args',
        '-r', 'in:=/camera/image_raw',
        '-r', 'out/compressed:=/camera/image_compressed'
    ]
)
```

Bandwidth Usage Table

Topic	Unoptimized	Optimized	Savings
`/scan` (360°, 10Hz)	~500 KB/s	~150 KB/s	70%
`/map` (full publish)	~2 MB/update	~500 KB/update	75%
`/camera/image_raw`	~30 MB/s	~2 MB/s (compressed)	93%
Total	~33 MB/s	~3 MB/s	91%

---

7. Network Testing Commands

Check Connectivity

bash

```bash
# List all topics (should see topics from both machines)
ros2 topic list

# Check topic info
ros2 topic info /scan
ros2 topic info /map

# Echo a topic to verify data is arriving
ros2 topic echo /scan --once
```

Monitor Bandwidth

```bash
# Check bandwidth usage per topic
ros2 topic bw /scan
ros2 topic bw /map
ros2 topic bw /odom

# Monitor all topics
ros2 topic bw --all
```

Check Latency

```bash
# Monitor topic rate and latency
ros2 topic hz /scan
ros2 topic delay /scan

# Check TF latency
ros2 run tf2_ros tf2_echo odom base_footprint
```

DDS Configuration Check

```bash
# Verify DDS implementation
echo $RMW_IMPLEMENTATION  # Should show: rmw_cyclonedds_cpp

# Check DDS domain
echo $ROS_DOMAIN_ID  # Should match on both machines

# Run ROS2 doctor
ros2 doctor --report
```

Network Diagnostics

```bash
# Test network connection between machines
ping <robot_ip_address>

# Check network speed
iperf3 -c <robot_ip_address>

# Monitor network traffic
sudo tcpdump -i <interface> udp port 7400-7500
```

---

8. Firewall Configuration

Ubuntu/Debian Firewall Setup

```bash
# Allow CycloneDDS ports
sudo ufw allow 7400:7500/udp
sudo ufw allow 7400:7500/tcp

# Allow multicast
sudo ufw allow from 224.0.0.0/4

# Reload firewall
sudo ufw reload

# Check status
sudo ufw status verbose
```

Required Ports

Port Range	Protocol	Purpose
7400-7500	UDP	CycloneDDS discovery and data
7400-7500	TCP	CycloneDDS reliable transport
224.0.0.0/4	Multicast	DDS multicast discovery

---

9. Troubleshooting

Problem: High Latency

Symptoms:

- TF transforms are delayed
- Map updates are slow
- `transform_timeout` warnings

Solutions:

```yaml
# In slam_toolbox config
transform_timeout: 2.0  # Increase timeout
tf_buffer_duration: 120.0  # Increase buffer
map_update_interval: 10.0  # Reduce update frequency
```

Problem: Lost Packets / Incomplete Maps

Symptoms:

- Gaps in the map
- "No laser scan received" warnings
- Inconsistent loop closures

Solutions:

```xml
<!-- In cyclonedds.xml -->
<SocketReceiveBufferSize min="20MB"/>  <!-- Increase buffer -->
<SocketSendBufferSize min="20MB"/>

<!-- Add reliability settings -->
<Reliability>
  <MaxRetransmitPeriod>2s</MaxRetransmitPeriod>
  <HeartbeatPeriod>100ms</HeartbeatPeriod>
</Reliability>
```

Problem: Topics Not Visible

Symptoms:

- `ros2 topic list` doesn't show remote topics
- Nodes can't see each other

Solutions:

```bash
# 1. Check ROS_DOMAIN_ID matches
echo $ROS_DOMAIN_ID  # Run on both machines

# 2. Check network connectivity
ping <other_machine_ip>

# 3. Check firewall
sudo ufw status
sudo ufw allow 7400:7500/udp

# 4. Verify CycloneDDS config
echo $CYCLONEDDS_URI
cat $HOME/cyclonedds.xml

# 5. Check multicast routing
ip route show
```

Problem: High Bandwidth Usage

Symptoms:

- Network congestion
- Slow video streams
- High CPU on network interface

Solutions:

```yaml
# Reduce data volume
throttle_scans: 5  # Process every 5th scan
resolution: 0.15  # Coarser map
map_update_interval: 15.0  # Less frequent updates

# Disable non-essential topics
debug_logging: false
visualize_trajectory: false
```

Problem: Map Not Updating

Symptoms:

- Map stays static
- "Waiting for scan" messages

Solutions:

```bash
# 1. Check scan topic is publishing
ros2 topic hz /scan

# 2. Check TF tree
ros2 run tf2_tools view_frames
evince frames.pdf

# 3. Verify SLAM node is running
ros2 node list | grep slam

# 4. Check SLAM node logs
ros2 node info /slam_toolbox

# 5. Restart SLAM node with verbose logging
ros2 launch your_package slam.launch.py --log-level DEBUG
```

Common Error Messages

Error	Cause	Solution
`transform_timeout exceeded`	Network latency too high	Increase `transform_timeout`
`No laser scan received`	`/scan` topic not visible	Check firewall and ROS_DOMAIN_ID
`Controller manager not available`	Services not discovered	Increase `SPDPInterval` in cyclonedds.xml
`TF lookup would require extrapolation`	Clock synchronization issue	Ensure `use_sim_time` matches on all nodes

---

10. Performance Tuning Guide

Low Bandwidth Network (< 1 Mbps)

```yaml
throttle_scans: 10
map_update_interval: 30.0
resolution: 0.20
transform_publish_period: 0.5
debug_logging: false
visualize_trajectory: false
```

Medium Bandwidth Network (1-10 Mbps)

```yaml
throttle_scans: 3
map_update_interval: 5.0
resolution: 0.10
transform_publish_period: 0.2
debug_logging: false
visualize_trajectory: true
```

High Bandwidth Network (> 10 Mbps)

```yaml
throttle_scans: 1
map_update_interval: 2.0
resolution: 0.05
transform_publish_period: 0.1
debug_logging: true
visualize_trajectory: true
```

Quick Reference Commands

bash

```
# Start on robot
export ROS_DOMAIN_ID=42
export CYCLONEDDS_URI=file://$HOME/cyclonedds.xml
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
ros2 launch rover_bringup robot.launch.py

# Start on base station
export ROS_DOMAIN_ID=42
export CYCLONEDDS_URI=file://$HOME/cyclonedds.xml
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
ros2 launch rover_bringup slam.launch.py

# Monitor system
ros2 topic bw --all  # Bandwidth
ros2 topic hz /scan  # Rate
ros2 doctor --report  # Health check
```

Summary Checklist

- [ ] Create `cyclonedds.xml` on both machines
- [ ] Update SLAM parameters for network optimization
- [ ] Export CycloneDDS environment variables
- [ ] Set matching `ROS_DOMAIN_ID` on both machines
- [ ] Configure firewall to allow ports 7400-7500
- [ ] Test network connectivity
- [ ] Monitor bandwidth usage
- [ ] Tune parameters based on network performance

Document Version: 1.0
Last Updated: 2025-12-26
Author: Claude (Anthropic)